Lecture 8 - Solutions

Course: DM571 / DM889 Software Engineering
Model answers. Other wording is fine if it means the same. All Git outputs below are from real runs, shortened. Commit ids and dates will be different for you.
Code: `solution/campusbike.py` (the file at the end of Part 4).

Part 1 - Set up

- A's fork shows `campusbike.py`, `README.md`, the `exercise` and `solution` folders, and the line "forked from" the course repository. Both clones print `Cloning into 'campusbike-git'...` and contain the same files.
- Adding B as a collaborator on the fork works: a fork of a public repository has its own settings and collaborators, separate from the course repository.
- Clone with the **HTTPS** URL. The password prompt needs a **token**, not the GitHub password. If a browser window opens instead (Git Credential Manager, usual on Windows), logging in there is enough.
- B needs a **classic** token with the **repo** box ticked. A fine-grained token cannot push to a repository where you are only a collaborator.
- Cloned the course repository by mistake? `git remote -v` shows the course URL, and `git push` fails with `403` ("Permission ... denied").
  `git remote set-url origin https://github.com/A-USERNAME/campusbike-git.git` points the clone at A's fork. Your commits stay; nothing needs to be cloned again.
- No pull request in Parts 1-3, and never one to the course repository. In Part 4 the base repository is A's fork.

Part 2 - First conflict on main

B's push is rejected, because GitHub has A's commit and B does not:

```
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to '.../campusbike-git.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. ...
```

`git pull --no-rebase` fetches A's commit and tries to merge it:

```
Auto-merging campusbike.py
CONFLICT (content): Merge conflict in campusbike.py
Automatic merge failed; fix conflicts and then commit the result.
```

The file now contains:

```
# Longest rental without a late fee, in hours
<<<<<<< HEAD
MAX_RENTAL_HOURS = 2
=======
MAX_RENTAL_HOURS = 4
>>>>>>> bb86d3893a205ab42a6a4e4c79e07c18b65d0f14
```

- `HEAD` is B's own version. The long id after `>>>>>>>` is A's commit.
- Git cannot choose, because both changed the **same line**. Only people can decide which value is right. Any agreed value is correct. In our run the pair agreed on 4.
- Running the file with markers left in stops at once with a `SyntaxError`. In our run: `SyntaxError: invalid syntax`, pointing at `<<<<<<< HEAD`. Another run gave `invalid decimal literal` at the `>>>>>>>` line. The exact text depends on the commit id.
- After the fix: `Max rental time: 4 hours`, `Price for 4 hours: 40 DKK`. Then `git add`, `git commit -m "Merge: agree on 4 hours"` and `git push` work.
- A's `git pull --no-rebase` in Step 6 says `Fast-forward`: A had nothing new, so Git just moves A's `main` forward. No second merge commit.

Part 3 - Branches that merge cleanly

The first push of each branch:

```
 * [new branch]      price -> price
branch 'price' set up to track 'origin/price'.
```

A's merge of both branches:

```
$ git merge price
Updating 79618f3..f1e9767
Fast-forward
$ git merge origin/late-fee -m "Merge late-fee"
Auto-merging campusbike.py
Merge made by the 'ort' strategy.
```

The file now has both changes, and the program prints `Price for 2 hours: 24 DKK` and `Price for 4 hours: 48 DKK`:

```
MAX_RENTAL_HOURS = 4

# Price per hour, in DKK
PRICE_PER_HOUR = 12

# Extra fee if the bike is kept too long, in DKK
LATE_FEE = 40
```

- **Why no conflict:** the two changes are on **different lines**, with unchanged lines between them. Git combines them by itself.
- `price` was a **fast-forward**: `main` had no new commits since the branch started, so Git just moved `main` forward. No merge commit.
- Good to know: if two changes are on lines **right next to each other**, Git also calls it a conflict. That is why the settings in `campusbike.py` have a comment and an empty line between them.

Part 4 - A pull request with a conflict

- **PR 1** (`start-fee` into `main` of A's fork) merges at once. GitHub makes a merge commit called `Merge pull request #1 from A-USERNAME/start-fee`. It always makes a merge commit, even when a fast-forward would be possible.
- **Order:** A merges PR 1 only after B has committed (Step 1). If A merges first and B starts Step 1 after that, B's branch already has the start fee. B's edit then simply replaces it: PR 2 merges with no conflict, and the start fee is gone without anyone deciding (tested).
- **PR 2** (`first-hour-free`): before B clicks Create pull request, GitHub may say "Can't automatically merge". Creating it is still fine. It cannot merge: `main` now has A's version of the same line. We reproduced GitHub's test merge with Git:

```
CONFLICT (content): Merge conflict in campusbike.py
Automatic merge failed; fix conflicts and then commit the result.
```

B's `git pull --no-rebase origin main` on the branch gives the same conflict:

```
 * branch            main       -> FETCH_HEAD
Auto-merging campusbike.py
CONFLICT (content): Merge conflict in campusbike.py
```

```
<<<<<<< HEAD
    price = (hours - 1) * PRICE_PER_HOUR
=======
    price = 5 + hours * PRICE_PER_HOUR
>>>>>>> 80446f26ab86f8aae7457d15a742b277ad87f53f
```

- `HEAD` is now B's **branch**. The part below is `main` from GitHub, with A's start fee.
- Any agreed line is correct. In our run the pair kept the start fee. The file is `solution/campusbike.py`, and it prints `Price for 2 hours: 29 DKK` and `Price for 4 hours: 53 DKK`.
- After B's `git push`, PR 2 has no conflicts any more, because the branch now contains `main`. A (the owner) clicks Merge, which makes `Merge pull request #2 from A-USERNAME/first-hour-free`.
- Why `origin main`? Plain `git pull --no-rebase` on the branch pulls the branch itself from GitHub: `Already up to date`, and the pull request still has conflicts.

Part 5 - Read the history

`git log --oneline --graph` at the end (the same for A and B):

```
*   3df8ef0 Merge pull request #2 from A-USERNAME/first-hour-free
|\
| *   4574516 Merge main: agree on the start fee
| |\
| |/
|/|
* |   80446f2 Merge pull request #1 from A-USERNAME/start-fee
|\ \
| * | f865af5 Add a start fee of 5 DKK
|/ /
| * 6290b30 Make the first hour free
|/
*   572f503 Merge late-fee
|\
| * 32fab65 Lower late fee to 40 DKK
* | f1e9767 Raise price to 12 DKK per hour
|/
*   79618f3 Merge: agree on 4 hours
|\
| * bb86d38 Set max rental time to 4 hours
* | f558af9 Set max rental time to 2 hours
|/
* 1f62325 Add exercise and solution
* d594229 Use py on Windows in README
* fce138c Add README
* ec00c64 Add campusbike.py
```

`git blame campusbike.py` (some lines; the date and time are cut to `...`):

```
^ec00c64 (Oliver Larsen ...  4) # Longest rental without a late fee, in hours
bb86d389 (Anna Holm     ...  5) MAX_RENTAL_HOURS = 4
f1e97673 (Anna Holm     ...  8) PRICE_PER_HOUR = 12
32fab659 (Ben Larsen    ... 11) LATE_FEE = 40
f865af58 (Anna Holm     ... 16)     price = 5 + hours * PRICE_PER_HOUR
^ec00c64 (Oliver Larsen ... 17)     if hours > MAX_RENTAL_HOURS:
```

(Anna Holm is Person A, Ben Larsen is Person B.)

1. **Who changed which line.** `MAX_RENTAL_HOURS`: A, `PRICE_PER_HOUR`: A, `LATE_FEE`: B, the `price = ...` line: A (start fee). All other lines come from the first commit (`Add campusbike.py`, the course file); the `^` in front marks the commit where the history starts. The other three starting commits do not show up: they changed other files (the README and the PDFs). B made the merge commit in Part 2, but blame shows **A** on the `MAX_RENTAL_HOURS` line: blame shows who wrote the **text** of the line, and B kept A's line unchanged. Had they agreed on 2, it would show B's commit; with a new value, for example 5, it would show B's merge commit.
2. **Where the merges are.** Every `*` with a split (`|\`) under it is a merge commit with two parents. There are five: `Merge: agree on 4 hours` (Part 2, B), `Merge late-fee` (Part 3, A), `Merge pull request #1` (GitHub), `Merge main: agree on the start fee` (Part 4, B, on B's branch) and `Merge pull request #2` (GitHub). GitHub made the two called "Merge pull request". The author is the person who clicked Merge.
3. **Why side branches.** A side line means two lines of work started from the same commit and went on **in parallel**, until a merge joined them again. `price` has no side line because its merge was a fast-forward: nothing else happened on `main` in between. Part 4 has the most lines: B merged `main` into the branch, and then GitHub merged the branch into `main`.

On one laptop (fallback): the same conflicts and the same clean merge, without a pull request. The final graph has three splits and joins, with the branch names as labels:

```
*   f11b373 (HEAD -> main) Merge first-hour-free: agree on the start fee
|\
| * 430325a (first-hour-free) Make the first hour free
* | 2437950 (start-fee) Add a start fee of 5 DKK
|/
*   4d93284 Merge late-fee
|\
| * e4da1c1 (late-fee) Lower late fee to 40 DKK
* | 316db1a (price) Raise price to 12 DKK per hour
|/
*   94af088 Merge student-b: agree on 4 hours
|\
| * dd00806 (student-b) Set max rental time to 2 hours
* | bb86d38 Set max rental time to 4 hours
|/
* 1f62325 (origin/main, origin/HEAD) Add exercise and solution
* d594229 Use py on Windows in README
* fce138c Add README
* ec00c64 Add campusbike.py
```

- The final file is the same as in the GitHub flow.
- Wrong order (branch made **after** the commit on `main`): the branch already has the new value, so `git merge` says `Fast-forward` and there is no conflict (tested).

What to check

Part	Good	Common mistake
2	All three marker lines deleted, one agreed value left; `git add`, then `git commit -m`	Markers committed (Git allows it), or "Committing is not possible because you have unmerged files"
3	Both 12 and 40 in the file on `main`, pushed	Merging before B pushed: `merge: origin/late-fee - not something we can merge`
4	Both pull requests in A's fork, conflict fixed locally	Pull request to the course repository, or conflict fixed with the web editor
5.1	Blame shows who wrote the text of a line, not who merged	"B, because B made the merge"
5.2	Five merge commits, two made by GitHub	Counting every commit with a branch name
5.3	Parallel work from one commit; fast-forward has no side line	"Branches are copies of the repository"